ASSIGNMENT 11.3
2303A51649

Task 1: Smart Contact Manager (Arrays & Linked Lists)

1.1 AI Prompt Used

```
Generate a Python program for a Contact Manager using arrays (Python
lists).
It should allow the user to:
1. Add a contact (name and phone number)
2. Search for a contact
3. Delete a contact
Display contacts after each operation.
```

CODE :

```
contacts = []

def add_contact():
    name = input("Enter name: ")
    phone = input("Enter phone number: ")
    contacts.append([name, phone])
    print("Contact added!")
```

```
def search_contact():
    name = input("Enter name to search: ")
    for contact in contacts:
        if contact[0] == name:
            print("Found:", contact)
            return
    print("Contact not found")

def delete_contact():
    name = input("Enter name to delete: ")
    for contact in contacts:
        if contact[0] == name:
            contacts.remove(contact)
            print("Contact deleted")
            return
    print("Contact not found")

while True:
    print("\n1.Add 2.Search 3.Delete 4.Show 5.Exit")
    choice = input("Enter choice: ")

    if choice == '1':
        add_contact()
```

```
    elif choice == '2':
        search_contact()
    elif choice == '3':
        delete_contact()
    elif choice == '4':
        print("Contacts:", contacts)
    elif choice == '5':
        break
```

Output :

```
1.Add 2.Search 3.Delete 4.Show 5.Exit
Enter choice: 1
Enter name: Ram
Enter phone number: 9876543210
Contact added!
Enter choice: 4
Contacts: [['ball', '9876543210']]
Enter choice: 2
Enter name to search: Ram
```

1.2 AI Prompt Used

```
Generate a Python program implementing a Contact Manager using Linked
List.
Include add, search and delete operations.
```

Code :

```
class Node:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None


class ContactList:
    def __init__(self):
        self.head = None
```

```
def add_contact(self, name, phone):
    new_node = Node(name, phone)
    new_node.next = self.head
    self.head = new_node
    print("Contact added")


def search(self, name):
    temp = self.head
    while temp:
        if temp.name == name:
            print("Found:", temp.name, temp.phone)
            return
        temp = temp.next
```

```
    print("Contact not found")


def delete(self, name):
    temp = self.head
    prev = None

    while temp:
        if temp.name == name:
            if prev:
                prev.next = temp.next
            else:
                self.head = temp.next
            print("Contact deleted")
            return
```

```
            prev = temp
            temp = temp.next

        print("Contact not found")

    def display(self):
        temp = self.head
        while temp:
            print(temp.name, temp.phone)
            temp = temp.next


cl = ContactList()
```

cl.add_contact("balli","9876543210")

cl.add_contact("Arjun","9123456780")

cl.display()

cl.search("balli")

cl.delete("Arjun")

cl.display()

Output

```
Contact added
Contact added
Arjun 9123456780
Balli 9876543210

Found: balli 9876543210

Contact deleted

Balli 9876543210
```

Task 2: Library Book Search System (Queue & Priority Queue)

AI Prompt

```
Generate a Python program implementing a Queue and Priority Queue
for library book requests where faculty have higher priority than students.
```

CODE :

```
from collections import deque

queue = deque()

def enqueue():
    name = input("Enter requester name: ")
    queue.append(name)

def dequeue():
    if queue:
        print("Processing request of:", queue.popleft())
    else:
        print("Queue empty")

enqueue()
enqueue()
dequeue()
```

OUTPUT :
Enter requester name: BALLI
Enter requester name: Arjun
Processing request of: BALLI

2B Priority Queue Implementation

Code

```
import heapq

pq = []

def add_request(name, role):
    if role == "faculty":
        priority = 1
```

```
    else:
        priority = 2
    heapq.heappush(pq, (priority, name))

def process_request():
    if pq:
        print("Processing:", heapq.heappop(pq)[1])

add_request("Student_Ram","student")
add_request("Prof_Sharma","faculty")
add_request("Student_Arjun","student")

process_request()
```

process_request()

process_request()

Output

```
Processing: Prof_Sharma
Processing: Student_Arjun
Processing: Student_GJAR
```

Task 3: Emergency Help Desk (Stack)

AI Prompt

```
Generate a Python program implementing a stack with push, pop, peek
and functions to check if stack is empty.
Simulate 5 help desk tickets.
```

CODE :

```
stack = []

def push(ticket):
    stack.append(ticket)
    print(ticket,"added")

def pop():
    if stack:
        print("Resolved:", stack.pop())
    else:
        print("Stack empty")

def peek():
    if stack:
        print("Current ticket:", stack[-1])

def is_empty():
    return len(stack) == 0

push("Ticket1")
push("Ticket2")
push("Ticket3")
push("Ticket4")
push("Ticket5")

peek()

pop()
```

```
pop()
```

Output

```
Ticket1 added
Ticket2 added
Ticket3 added
Ticket4 added
Ticket5 added

Current ticket: Ticket5

Resolved: Ticket5
Resolved: Ticket4
```

Task 4: Hash Table (Collision Handling)

AI Prompt

```
Generate a Python hash table with insert, search and delete functions
using chaining for collision handling.
```

Code

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        self.table[index].append((key,value))
        print("Inserted:", key)

    def search(self, key):
        index = self.hash_function(key)
        for k,v in self.table[index]:
            if k == key:
```

```
                print("Found:", v)
                return
        print("Not found")

    def delete(self, key):
        index = self.hash_function(key)
        for i,(k,v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                print("Deleted:", key)
                return
        print("Not found")

ht = HashTable()

ht.insert("Ram",9876543210)
ht.insert("Arjun",9123456780)

ht.search("Ram")
ht.delete("Arjun")
```

Output

```
Inserted: SIDDHU
Inserted: Arjun
Found: 9876543210
Deleted: Arjun
```

Task 5: Campus Resource Management System

Code

```
from collections import deque

orders = deque()

def place_order(order):
    orders.append(order)
    print("Order placed:", order)

def serve_order():
    if orders:
        print("Serving:", orders.popleft())
    else:
        print("No orders")

place_order("Burger")
place_order("Pizza")
place_order("Coffee")

serve_order()
serve_order()
```

Output

```
Order placed: Burger
Order placed: Pizza
Order placed: Coffee
Serving: Burger
Serving: Pizza
```